Project Work and Future Enhancement

Modern Airline Logistics and Sustainability Management System

Pranali Paygonda Patil, 25BCE2957, B.Tech Computer Science and Engineering, VIT SCOPE, AY 2026–27

Project purpose

This project develops a database-backed system for organizing airline logistics in one connected model. Airline work depends on accurate links among aircraft, flights, crew, passengers, cargo and operational events. When those records sit in separate lists, staff may face inconsistent information, slower coordination and limited visibility into delays or environmental impact. The project combines a relational database with a browser console so users can review and maintain key records in a shared operational view.

Work completed

The Review 0 work defines a normalized relational design with nine entities: AIRCRAFT, FLIGHT, CREW, FLIGHT_CREW, LOYALTY_TIER, PASSENGER, SPEC_CARGO, EMISSIONS and DISRUPTION. FLIGHT acts as the operational anchor. Primary and foreign keys preserve record identity and relationships. UNIQUE, NOT NULL, CHECK and DEFAULT constraints help prevent invalid or duplicate data. The many-to-many relationship between flights and crew is represented through FLIGHT_CREW, which also records shift hours.

The database coursework demonstrates table creation, inserts, updates and deletions, along with SELECT statements, single-row and aggregate functions, GROUP BY, HAVING, subqueries and joins. These operations support both routine record management and questions that combine information across related tables.

Sustainability motivation

The sustainability focus comes from making environmental impact visible alongside flight operations. EMISSIONS stores fuel consumed, CO_2 recorded and the time of each log, linked to a flight. This creates a basis for later route or aircraft comparisons. The project records and organizes sustainability data; it does not claim that the prototype has reduced emissions.

Implementation and practical use

The current application serves a single-page HTML, CSS and JavaScript interface through a local FastAPI service. Python's python-oracledb driver connects to Oracle XE in Thin mode. The browser requests JSON data from the API, and create, update and delete actions are committed to Oracle. The README describes running the service with Uvicorn at 127.0.0.1:8001 after the local Oracle credentials and service name are configured.

Operational features

- The console provides views for flights, aircraft, crew, passengers, specialized cargo, emissions and disruptions. Loyalty tiers serve as a passenger lookup.
- Users can search records, filter by status, export CSV files, inspect record details and use forms to add or edit records. The interface refreshes Oracle data every five seconds.
- Cargo records include category, weight, required temperature, current temperature and tracking status. Disruption records capture a flight, reason, delay duration, event time and status.
- Emission records link fuel and CO_2 values to a flight. SQL joins and grouped summaries can use these links to compare routes or investigate patterns.
- Account registration and sign-in protect the operational API. Passwords use scrypt hashes, and sessions use a signed HttpOnly cookie with a 12-hour expiry.

Business and operational relevance

A shared flight-linked record can help operations staff review aircraft assignments and disruption status without reconciling separate lists. Cargo coordinators can compare a shipment's required and current temperature. Sustainability analysts can prepare route-level comparisons from fuel and CO_2 logs. Managers can group disruption events by reason or status to identify recurring causes for further investigation. These are potential uses of the data model, not measured improvements from a live airline deployment.

Current scope and limitations

The Review 0 document lists MySQL 8.0, while the current runnable code and README use Oracle XE. The database design describes FLIGHT_CREW, but the browser application does not provide a dedicated screen for managing crew-to-flight assignments. User accounts currently share the same access level, the README limits use to localhost coursework, and Oracle key generation must be configured for add forms when an ID is omitted. Bundled browser sample records and the overview chart are illustrative; use live Oracle data for operational reporting.

Future enhancements

The next stage should improve data quality and operational usefulness before expanding deployment. Enhancements below are proposals and are not part of the current implementation.

1. Strengthen validation and access control

Add role-based permissions for operations, cargo and sustainability users. Validate date order, numeric ranges, status values and required fields in both the interface and API. Define an explicit identity strategy for every table and retain an audit history of who changed each record and when. These changes would make the system safer to use and easier to review.

2. Add live cold-chain monitoring

Extend cargo records to hold timestamped sensor readings rather than a single current temperature. Let staff configure acceptable ranges by cargo type, raise an alert when a reading crosses a limit, and record acknowledgement and resolution. A pilot could begin with one shipment category and compare sensor readings with the existing manual workflow.

3. Build sustainability analysis

Create dashboards for fuel and CO_2 by flight, route and aircraft. Record the source and method used for each emissions value, and add route distance or payload information before calculating intensity measures. This would make comparisons more meaningful and help analysts distinguish a higher total caused by a longer route from a less efficient operation.

4. Improve disruption and fleet planning

Add summaries for delay duration and disruption reason, then connect those findings with schedule and aircraft data. Later versions could incorporate weather or air-traffic feeds to help staff see developing risks. Any recommendation feature should explain which records support it and leave operational decisions with authorized staff.

5. Prepare for reliable deployment

Before use beyond a local demonstration, add protected secret storage, encrypted network traffic, structured logs, monitoring, backups and recovery procedures. Test database constraints and API behavior with representative cases, document setup for a clean environment, and integrate other systems only through defined and access-controlled interfaces.

Conclusion

The project establishes a practical foundation for connected airline logistics records and introduces sustainability and cold-chain data into the operational model. Its strongest next steps are reliable validation, traceable live cargo readings and clearly sourced emissions analysis. With these improvements, the prototype could provide more dependable information for airline coordination while preserving a clear record of how each operational decision is supported.

Project basis: Review 0 report, current README, server.py and index.html.